Thứ ngày tháng năm

Problem 1 : 2,0
Problem 2 : 2,5
Problem 3 : 2,25
Problem 4 : 2,25

Võ Lâm

2,00 Problem 1: Singly linked list - In place Reversal and pointer tracing

1,25 task 1: write a algo/func in pseudo code to reverse the singly linked list including 3 pointers (prev, curr, next) to complete

```
function reverselist(head):
    prev = Null
    curr = head
    next = Null

    While (curr is not Null:)
        next = curr.next
        curr.next = prev
        prev = curr
        curr = next

        return head
```


Scanned with CamScanner

Thứ ngày tháng năm

0.75 task 2: Draw a digram showing the position and values of 3 pointers after each iteration

Iteration	prev	curr	next
initial	Null	10	unitilized
1	10	25	25
2	25	40	40
3	40	15	15
4	15	Null	Null

Linked Structure ?

Final result: ?

2.5 Problem 2: Singly circular linked list:

head → [10] → [20] → [30] → head

4.25 task 1: write pseducode for the function InsertHead(head, newValue) to insert a new node cotaining new value at front and maintaing the intesity

Scanned with CamScanner

Thứ ngày tháng năm

```
function InsertHead(head, NewValue)

    newNode = allocate new Node(
                data = newValue, next = Null)

    If (head is Null):
        NewNode.next = newNode
        head = newnode
        return head

    tail = head
    while (tail.next != head)
        tail = tail.next

    newNode.next = head
    tail.next = newNode
    head = newNode

    return head
```

1,25 Task 2: Trace the execution step when

Scanned with CamScanner

Thứ ngày tháng năm

inserting newValue = 5

Step 1: create newNode * containing 5

Step 2: check if list is emty. Its not

Step 3: Set tail = head ([10]) and traverse to last note

3.1: check if tail.next = head. Its not
tail = [20]

3.2: Same as 3.1
tail = [30]

3.3: tail.next is equal to head. ~~terminate~~
break loop

Step 4: Set newNode.next = head
[5] ~~[5]~~.next points to [5]

Step 5: Set tail.next = newNode
[30].next points to [5]

Scanned with CamScanner

(2)

Thứ ngày tháng năm

Step 6: Set head = newNode
head points to [5]

Finally!

head → [5] → [10] → [20] → [30] → head

2.25 Problem 3: Doubly linked list - Insert before target and Edge cases

1.75 tas 1 : write pseudo code for Insert Befor(-head, target, newValue)

tas 3 : Explicitly handling these tow:

1. target is head node
2. target dont exist

answening both tast 1 and 3:

function Insert Before(head, target, newValue):
newNode = allocate new Node(
data = newValue, next = Null, prev = Null)

Scanned with CamScanner

Thứ ngày tháng năm

```
if head is Null:
    return head

if head.data == target:
newNode.next = head
    head.prev = newNode
    head = newNode
    return head

curr = head
while (curr is not Null and curr.data != target)
    curr = curr.next

If curr is Null:
    free(newNode)   RAII
    return head

newNode.next = curr
newNode.prev = curr.prev
curr.prev.next = newNode
```


Scanned with CamScanner

curr.prev = newNode

Return head

0,5 task 2: list the exact order of pointer assignments required and briefly explain why there is no memory leak or broken link.

~~List~~ listing:

Step 1: newNode.next = curr

Step 2: newNode.prev = curr.prev

Step 3: curr.prev.next = newNode

Step 4: curr.prev = newNode

Explaination: the core idea is to established the link from the new node to others before linking others to the new node

2,25 problem 4: Sorted linked list - In order Insertion

Scanned with CamScanner

Thứ ngày tháng năm

head → [2] → [5] → [8] → Null

1,00 Test 2: write function SortedInserted(head, val)

```
function SortInserted(head, val):
    newNode = allocate new Node(data = val,
                                next = Null)

    If head is Null or head.data >= val:
        nextNode.next = head
        head = newNode
        return head

    curr = head

    while curr.next is not Null and curr.next
        curr.next.data < val:
            curr.next = curr

    newNode.next = curr.next
```


Scanned with CamScanner

3

Thứ ngày tháng năm

curr.next = newNode

return head

1,25 Task 2: Step by step table detailing pointer iterations and condition evaluation when insert val = 6

Iteration	State	curr.next != Null	curr.next.data < 6
initial	curr = [2]	-	-
1	curr = [2]	true	true
2	curr = [5]	true	false
insertion	curr = [5]	-	-

Scanned with CamScanner